

# 解析几何题目生成系统

## 技术原理说明文档

结合 LLM 与智能体方法的实践项目

2026 年 6 月

## 目录

|          |                                |          |
|----------|--------------------------------|----------|
| <b>1</b> | <b>项目概述</b>                    | <b>2</b> |
| 1.1      | 背景                             | 2        |
| 1.2      | 目标                             | 2        |
| 1.3      | 技术栈                            | 2        |
| <b>2</b> | <b>系统架构</b>                    | <b>2</b> |
| 2.1      | 整体架构                           | 2        |
| 2.2      | 模块依赖                           | 3        |
| 2.3      | 双路径生成策略                        | 3        |
| <b>3</b> | <b>核心数据模型</b>                  | <b>4</b> |
| 3.1      | ConicParams —圆锥曲线参数            | 4        |
| 3.2      | Point —二维点                     | 5        |
| 3.3      | Line —直线 ( $ax + by + c = 0$ ) | 5        |
| 3.4      | Problem —题目对象                  | 6        |
| <b>4</b> | <b>题目生成算法</b>                  | <b>7</b> |
| 4.1      | 总体流程                           | 7        |
| 4.2      | 椭圆焦点弦—算法示例                     | 7        |
| 4.3      | 题型覆盖                           | 8        |
| 4.4      | 可解性保证                          | 8        |
| <b>5</b> | <b>精确配图渲染</b>                  | <b>9</b> |
| 5.1      | 设计原则                           | 9        |
| 5.2      | 渲染流程                           | 9        |
| 5.3      | 参数方程绘制                         | 9        |
| 5.4      | 坐标系自适应                         | 10       |
| 5.5      | 配色方案                           | 10       |

|                               |           |
|-------------------------------|-----------|
| <b>6 LaTeX 渲染引擎</b>           | <b>10</b> |
| 6.1 设计目标                      | 10        |
| 6.2 转换规则                      | 11        |
| 6.3 实现原理                      | 11        |
| <b>7 TUI 交互界面</b>             | <b>11</b> |
| 7.1 界面设计                      | 11        |
| 7.2 自然语言解析                    | 12        |
| 7.3 难度路由                      | 12        |
| <b>8 LLM 与智能体结合—AI 辅助开发实践</b> | <b>12</b> |
| 8.1 开发范式：人机协作                 | 12        |
| 8.2 AI 参与的具体环节                | 13        |
| 8.2.1 架构设计阶段                  | 13        |
| 8.2.2 数学推导阶段                  | 13        |
| 8.2.3 代码生成阶段                  | 14        |
| 8.2.4 UI 设计阶段                 | 14        |
| 8.2.5 测试阶段                    | 14        |
| 8.3 AI 辅助开发的工作流               | 14        |
| 8.4 AI 的优势与局限                 | 14        |
| 8.5 智能体应用场景                   | 15        |
| <b>9 测试与验证</b>                | <b>16</b> |
| 9.1 全题型测试                     | 16        |
| 9.2 测试结果                      | 16        |
| 9.3 输出文件                      | 17        |

## 1 项目概述

### 1.1 背景

解析几何是高中数学的核心内容，涵盖椭圆、双曲线、抛物线和极坐标等知识点。传统的出题方式依赖教师手动编写，存在以下问题：

- **人工成本高**：每道题需要手动推导参数、验证可解性、绘制配图
- **配图不精确**：手绘或简单绘图工具难以保证关键点位置与数学参数一致
- **题型覆盖有限**：难以系统性地覆盖从基础到竞赛的全部题型

### 1.2 目标

本系统旨在实现：

1. 用户指定知识点（椭圆/双曲线/抛物线/极坐标）与参数，**自动生成**可解的解析几何题目
2. 生成**格式严谨的 LaTeX** 题干与完整解答
3. 代码渲染**精确配图**，图中所有关键点、直线、曲线的位置与题干数学参数严格一致
4. 覆盖 **60+ 种题型**，从基础标准方程到高考压轴和竞赛难度

### 1.3 技术栈

| 组件   | 技术             | 用途             |
|------|----------------|----------------|
| 语言   | Python 3.12    | 主开发语言          |
| 数值计算 | NumPy          | 矩阵运算、方程求解、参数计算 |
| 绘图   | Matplotlib     | 参数方程精确绘图       |
| TUI  | Textual + Rich | 终端交互界面         |
| 打包   | PyInstaller    | 独立可执行文件        |

表 1: 技术栈

## 2 系统架构

### 2.1 整体架构

系统采用**双路径生成策略**，两条路径共享同一数据模型：

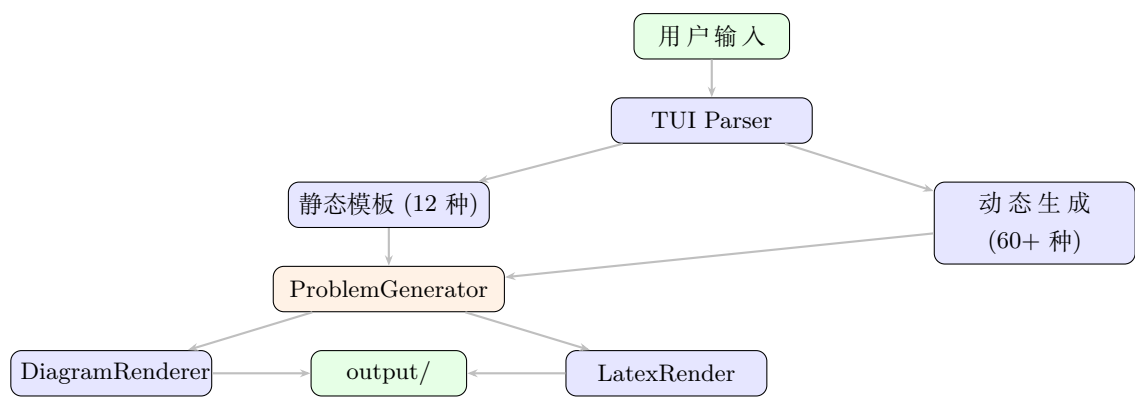


图 1: 系统整体架构图

2.2 模块依赖

run.py

统一入口

TUI 模式 → tui\_app.py

interactive\_generator.py (动态生成)  
diagram\_renderer.py (配图渲染)  
latex\_render.py (LaTeX转换)

CLI 模式 → interactive\_generator.py

problem\_generator.py (数据模型)  
diagram\_renderer.py  
latex\_render.py

批量模式 → main.py

problem\_generator.py  
diagram\_renderer.py  
create\_ppt.py

2.3 双路径生成策略

| 路径   | 模块                       | 题型数 | 用途        |
|------|--------------------------|-----|-----------|
| 静态模板 | problem_generator.py     | 12  | 批量生成、演示   |
| 动态生成 | interactive_generator.py | 60+ | 用户交互、指定参数 |

表 2: 双路径生成策略

Note

两条路径共享 Problem、ConicParams、Point、Line 等数据类，保证输出格式一致。

## 3 核心数据模型

### 3.1 ConicParams —圆锥曲线参数

```
1 @dataclass
2 class ConicParams:
3     """圆锥曲线参数"""
4     center: Tuple[float, float] = (0, 0) # 中心点
5     a: float = 1.0 # 半长轴 / 半实轴
6     b: float = 1.0 # 半短轴 / 半虚轴
7     c: float = 0.0 # 半焦距
8     e: float = 0.0 # 离心率
9     rotation: float = 0.0 # 旋转角度(弧度)
10
11     def __post_init__(self):
12         if self.c == 0:
13             self.c = np.sqrt(abs(self.a**2 - self.b**2))
14         if self.e == 0 and self.c > 0:
15             self.e = self.c / self.a
```

#### Args:

- center (Tuple[float, float]) —中心点坐标，默认 (0,0)
- a (float) —半长轴/半实轴，默认 1.0
- b (float) —半短轴/半虚轴，默认 1.0
- c (float) —半焦距，默认 0.0（自动计算）
- e (float) —离心率，默认 0.0（自动计算）
- rotation (float) —旋转角度（弧度），默认 0.0

#### 自动计算规则:

- 椭圆:  $c = \sqrt{a^2 - b^2}$ ,  $e = c/a$
- 双曲线:  $c = \sqrt{a^2 + b^2}$ ,  $e = c/a$
- 抛物线:  $a = p/2$ ,  $b = 0$ ,  $c = p/2$

#### Tip

$c$  和  $e$  在 `__post_init__` 中自动计算，避免手动推导错误。

### 3.2 Point — 二维点

```
1 @dataclass
2 class Point:
3     """ 二维点 """
4     x: float
5     y: float
6     label: str = ""
7
8     def distance_to(self, other: 'Point') -> float:
9         """ 计算到另一个点的距离 """
10        return np.sqrt((self.x - other.x)**2 + (self.y - other.y)**2)
```

#### Args:

- x (float) — x 坐标
- y (float) — y 坐标
- label (str) 一点的标签 (如 "F\_1", "P"), 默认空字符串

#### Methods:

- distance\_to(other) — 计算到另一个 Point 的欧几里得距离

### 3.3 Line — 直线 ( $ax + by + c = 0$ )

```
1 @dataclass
2 class Line:
3     """ 直线:  $ax + by + c = 0$  """
4     a: float
5     b: float
6     c: float
7     label: str = ""
8
9     def slope(self) -> float:
10        """ 计算斜率 """
11        return -self.a / self.b if self.b != 0 else float('inf')
12
13    def distance_to_point(self, p: Point) -> float:
14        """ 计算点到直线的距离 """
15        return abs(self.a * p.x + self.b * p.y + self.c) / np.sqrt(self.a
16                               **2 + self.b**2)
```

#### Args:

- a, b, c (float) — 直线方程  $ax + by + c = 0$  的系数

- `label (str)` —直线的标签，默认空字符串

#### Methods:

- `slope()` —返回直线的斜率（垂直线返回 `inf`）
- `distance_to_point(p)` —返回点 `p` 到直线的距离

### 3.4 Problem —题目对象

```

1 @dataclass
2 class Problem:
3     """解析几何题目"""
4     title: str          # 题目标题
5     topic: str          # 知识点 (ellipse/hyperbola/parabola/
        polar)
6     difficulty: int     # 难度等级 (1-5)
7     problem_latex: str  # 题干 LaTeX
8     solution_latex: str # 解答 LaTeX
9     conic_params: ConicParams # 圆锥曲线参数
10    points: List[Point]  # 关键点 (焦点、顶点、交点)
11    lines: List[Line]    # 关键直线 (渐近线、准线、弦)
12    conic_type: str     # 曲线类型
13    answer: str         # 最终答案

```

#### Args:

- `title (str)` —题目的显示标题
- `topic (str)` —知识点，可选值: "ellipse", "hyperbola", "parabola", "polar"
- `difficulty (int)` —难度等级，范围 1-5
- `problem_latex (str)` —题干的 LaTeX 格式文本
- `solution_latex (str)` —解答的 LaTeX 格式文本
- `conic_params (ConicParams)` —圆锥曲线的数学参数
- `points (List[Point])` —关键点列表（焦点、顶点、交点等）
- `lines (List[Line])` —关键直线列表（渐近线、准线、弦等）
- `conic_type (str)` —曲线类型，用于配图渲染
- `answer (str)` —最终答案的 LaTeX 格式文本

## 4 题目生成算法

### 4.1 总体流程

每种题型的生成遵循统一流程：

1. **参数选取**—从预设集合中随机选取  $a, b$  (或  $p, r$ ) 等参数
2. **参数推导**—计算  $c, e$ , 确定焦点、顶点等关键点坐标
3. **几何构造**—根据题型构造直线、计算交点
4. **可解性验证**—检查判别式  $\Delta \geq 0$
5. **LaTeX 生成**—使用 f-string 模板嵌入计算结果
6. **解答生成**—包含完整的推导过程

### 4.2 椭圆焦点弦—算法示例

以”椭圆焦点弦”题型为例，说明完整的生成算法。

**输入：**半长轴  $a$ , 半短轴  $b$ , 弦斜率  $k$

**Step 1 —计算基础参数**

$$c = \sqrt{a^2 - b^2}, \quad e = \frac{c}{a}$$

左焦点  $F_1(-c, 0)$ , 右焦点  $F_2(c, 0)$ 。

**Step 2 —构造直线方程**

过  $F_1$  斜率为  $k$  的直线：

$$y = k(x + c)$$

**Step 3 —联立方程求交点**

将直线代入椭圆方程  $\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$ ：

$$\frac{x^2}{a^2} + \frac{k^2(x+c)^2}{b^2} = 1$$

整理得二次方程：

$$(b^2 + a^2k^2)x^2 + 2a^2ck^2x + (a^2c^2k^2 - a^2b^2) = 0$$

**Step 4 —判别式验证**

$$\Delta = (2a^2ck^2)^2 - 4(b^2 + a^2k^2)(a^2c^2k^2 - a^2b^2) \geq 0$$

#### Warning

若  $\Delta < 0$ , 说明直线与椭圆无交点, 需要调整参数重新生成。



Step 5 —韦达定理求交点

$$x_1 = \frac{-B + \sqrt{\Delta}}{2A}, \quad x_2 = \frac{-B - \sqrt{\Delta}}{2A}$$
$$y_1 = k(x_1 + c), \quad y_2 = k(x_2 + c)$$

Step 6 —计算弦长与面积

$$|PQ| = \sqrt{1 + k^2} \cdot |x_1 - x_2|$$
$$S_{\triangle OPQ} = \frac{1}{2} |x_1 y_2 - x_2 y_1|$$

4.3 题型覆盖

系统覆盖 4 大知识点、60+ 种题型：

| 知识点 | 基础       | 进阶                      | 竞赛/压轴                                              |
|-----|----------|-------------------------|----------------------------------------------------|
| 椭圆  | 标准方程、焦点  | 焦点弦、中点弦、焦半径、切线、斜率积、第二定义 | 定点证明、面积最值、离心率范围、极点极线、第三定义、光学性质、轨迹方程、蒙日圆、阿波罗尼斯圆     |
| 双曲线 | 标准方程、渐近线 | 焦点弦、中点弦、焦半径、切线、斜率积、第二定义 | 渐近线平行弦、焦点三角形面积、离心率范围、极点极线、光学性质、轨迹方程、等轴双曲线、蒙日圆、蝴蝶问题 |
| 抛物线 | 标准方程、焦点  | 焦点弦、中点弦、焦半径、切线、斜率积、第二定义 | 阿基米德三角形、定点证明、焦点弦性质、光学性质、轨迹方程、向量搭桥、蒙日圆              |
| 极坐标 | 坐标互化     | 直线与圆、焦半径、弦长比值、斜率积、定点    | 圆锥曲线极坐标方程、第二定义、面积最值、统一极坐标方程、参数方程应用                 |

表 3: 题型覆盖

4.4 可解性保证

每道题在生成过程中都经过数学验证：

- **判别式检查**： $\Delta \geq 0$  确保直线与曲线有实交点
- **参数约束**：椭圆  $a > b > 0$ ，双曲线  $a, b > 0$ ，抛物线  $p > 0$
- **整数优化**：从  $\{2, 3, 4, 5, 6\}$  等小整数集合中选取参数，优先产生整数或简单分数解

## 5 精确配图渲染

### 5.1 设计原则

配图渲染的核心原则是**精确性**：图中所有关键点、直线、曲线的位置必须与题干数学参数严格一致。

### 5.2 渲染流程

Step 1: 确定坐标范围

根据 `conic_params` 计算基础范围  
遍历所有 `Point` 确保不裁剪  
等比例缩放 (`set_aspect='equal'`)

Step 2: 绘制坐标系

网格线 (`alpha=0.3`)  
坐标轴 (带箭头)  
刻度标签

Step 3: 绘制曲线 (参数方程, 1000 采样点)

椭圆:  $x = a \cdot \cos(), y = b \cdot \sin()$   
双曲线:  $x = \pm a \cdot \cosh(t), y = b \cdot \sinh(t)$   
抛物线:  $x = y^2 / (2p)$   
极坐标圆:  $r = 2r \cdot \cos()$

Step 4: 绘制辅助线

渐近线 (紫色虚线)  
准线 (绿色点划线)  
切线/弦 (红色实线)

Step 5: 标注关键点

焦点 (金色填充 + 蓝色边框)  
顶点 (蓝色)  
交点 (蓝色)

Step 6: 导出 PNG (150 DPI, 白底)

### 5.3 参数方程绘制

所有曲线使用参数方程绘制, 采样 1000 个点:

椭圆:

```
1 theta = np.linspace(0, 2 * np.pi, 1000)
```

```

2 x = center[0] + a * np.cos(theta)
3 y = center[1] + b * np.sin(theta)

```

双曲线（右支）:

```

1 x_right = np.linspace(a, x_max, 500)
2 y_right_pos = b * np.sqrt((x_right / a)**2 - 1)
3 y_right_neg = -y_right_pos

```

抛物线:

```

1 y_vals = np.linspace(y_min, y_max, 500)
2 x_vals = y_vals**2 / (2 * p)

```

## 5.4 坐标系自适应

```

1 def _get_plot_range(self, problem):
2     """根据题目参数确定合适的坐标范围"""
3     # 1. 根据曲线类型计算基础范围
4     # 2. 遍历所有关键点，扩展范围
5     for point in problem.points:
6         x_range = (min(x_range[0], point.x - margin),
7                     max(x_range[1], point.x + margin))
8     # 3. 确保正方形比例
9     max_span = max(x_span, y_span)

```

## 5.5 配色方案

| 元素   | 颜色          | 色值                |
|------|-------------|-------------------|
| 曲线   | 蓝色          | #2E86AB           |
| 直线/弦 | 红色          | #E8451E           |
| 焦点   | 金色填充 + 蓝色边框 | #FFD700 / #2E86AB |
| 渐近线  | 紫色虚线        | #9B59B6           |
| 准线   | 绿色点划线       | #27AE60           |
| 切线   | 橙色          | #F39C12           |

表 4: 配色方案

## 6 LaTeX 渲染引擎

### 6.1 设计目标

将 LaTeX 数学表达式转换为 Unicode 字符，实现在终端中直接显示数学符号。

6.2 转换规则

| LaTeX 输入                | Unicode 输出     | 说明   |
|-------------------------|----------------|------|
| $\frac{a}{b}$           | a/b            | 分数   |
| $x^2$                   | x <sup>2</sup> | 上标   |
| $x_1$                   | x <sub>1</sub> | 下标   |
| $\sqrt{x}$              | √x             | 根号   |
| $\alpha, \beta, \theta$ | , ,            | 希腊字母 |
| $\times, \pm, \leq$     | ×, ±,          | 运算符  |
| $\triangle$             |                | 几何符号 |

表 5: LaTeX 到 Unicode 转换规则

6.3 实现原理

转换器使用正则表达式逐层处理 LaTeX 文本：

```
1 def latex_to_unicode(text: str) -> str:
2     """将 LaTeX 数学表达式转换为 Unicode 终端显示"""
3     text = text.replace("$$", "")          # 移除块级定界符
4     text = text.replace("$", "")          # 移除行内定界符
5     text = re.sub(r'\\text\{([~]*)\}', r'\1', text) # \text{...}
6     text = _convert_frac(text)             # \frac{a}{b}
7     text = _convert_sqrt(text)             # \sqrt{...}
8     text = _convert_superscript(text)      # ^{...}
9     text = _convert_subscript(text)        # _{...}
10    # 替换希腊字母、运算符...
11    return text.strip()
```

Tip

对于  $\frac{\frac{a}{b}}{c}$  等嵌套结构，使用循环迭代处理（最多 20 次），每次替换最内层的  $\frac{\dots}{\dots}$ 。

7 TUI 交互界面

7.1 界面设计

TUI 基于 Textual 框架，参考 DeepSeek / Claude Code 的交互形式：

- **Chat 风格输入**：用户在底部输入框输入自然语言
- **可拖拽分栏**：左侧对话历史 + 右侧配图预览

- **侧边栏**：显示所有快捷命令和题型说明
- **暖色调主题**：cream (#faf9f5) + coral (#cc785c) 配色方案

7.2 自然语言解析

parse\_user\_input() 函数解析用户输入：

| 用户输入       | 解析结果                                            |
|------------|-------------------------------------------------|
| 椭圆 a=5 b=3 | {topic: "ellipse", a: 5, b: 3, type: "basic"}   |
| 抛物线竞赛      | {topic: "parabola", type: "random_competition"} |
| random     | {topic: "random", type: "random"}               |

表 6: 自然语言解析示例

7.3 难度路由

| 用户输入        | 映射            |
|-------------|---------------|
| 基础          | basic         |
| 进阶          | chord         |
| 竞赛 / 压轴 / 难 | 自动从高难度题型中随机选择 |

表 7: 难度路由

8 LLM 与智能体结合—AI 辅助开发实践

8.1 开发范式：人机协作

本项目采用**人机协作**的开发范式，而非传统的纯手写或纯 AI 生成。核心理念是：

**人类把控方向，AI 处理细节。**

具体来说：

- **人类负责**：需求定义、架构决策、质量把关、最终验证
- **AI (Claude) 负责**：数学推导、代码实现、模板生成、测试编写

Note

这种协作模式的关键在于**迭代反馈**：AI 生成初稿 → 人类审查 → AI 修正 → 人类验证。

## 8.2 AI 参与的具体环节

### 8.2.1 架构设计阶段

**人类输入：**描述项目目标——”生成解析几何题目，带精确配图，支持 TUI 交互”

**AI 输出：**

- 模块划分方案(problem\_generator.py + interactive\_generator.py + diagram\_renderer.py + latex\_render.py + tui\_app.py)
- 数据模型设计 (Problem, ConicParams, Point, Line 四个 dataclass)
- 接口定义 (generate(topic, difficulty) 统一入口)

**人类决策：**接受双路径方案，但要求动态生成支持更多题型。

### 8.2.2 数学推导阶段

这是 AI 最有价值的参与环节。每种题型的数学推导过程复杂，涉及：

- 联立方程组求解
- 韦达定理应用
- 判别式计算
- 面积公式推导
- 离心率范围分析

**以”椭圆焦点弦”为例，AI 的推导过程：**

人类：”生成椭圆焦点弦题，过左焦点斜率为k的直线与椭圆交于P、Q”

AI 推导：

1. 直线方程：  $y = k(x + c)$
2. 代入椭圆：  $x^2/a^2 + k^2(x+c)^2/b^2 = 1$
3. 整理：  $(b^2 + a^2k^2)x^2 + 2a^2ck^2x + a^2c^2k^2 - a^2b^2 = 0$
4. 判别式：  $\Delta = (2a^2ck^2)^2 - 4(b^2+a^2k^2)(a^2c^2k^2-a^2b^2)$
5. 韦达定理：  $x_1+x_2 = -2a^2ck^2/(b^2+a^2k^2)$
6. 弦长：  $|PQ| = \sqrt{(1+k^2)} \cdot |x_1-x_2|$
7. 面积：  $S = \frac{1}{2}|x_1y_2 - x_2y_1|$

AI 代码：

```
A_coeff = b**2 + a**2 * k**2
B_coeff = 2 * a**2 * c * k**2
C_coeff = a**2 * c**2 * k**2 - a**2 * b**2
```

```
discriminant = B_coeff**2 - 4 * A_coeff * C_coeff
x1 = (-B_coeff + np.sqrt(discriminant)) / (2 * A_coeff)
```

**人类验证：**检查数学推导的正确性，确认判别式逻辑。

### 8.2.3 代码生成阶段

AI 为每种题型生成三个部分：

1. **参数计算代码**—NumPy 数值计算
2. **LaTeX 题干模板**—f-string 格式，嵌入计算结果
3. **LaTeX 解答模板**—包含完整推导过程

### 8.2.4 UI 设计阶段

AI 参与了 TUI 的完整设计：

- **布局方案：**参考 DeepSeek/Claude Code 的 Chat 风格
- **配色系统：**cream + coral 暖色调方案（参考 Anthropic DESIGN.md）
- **交互逻辑：**自然语言解析、难度路由、实时渲染
- **Textual 组件：**SplitBar 可拖拽分栏、RichLog 聊天记录

### 8.2.5 测试阶段

AI 生成了 `test_all_types.py` 测试脚本，覆盖：

- 所有 60 种题型的生成测试
- 题干/解答/答案的非空验证
- LaTeX 渲染验证
- 配图文件生成验证
- 关键点存在性验证

## 8.3 AI 辅助开发的工作流

## 8.4 AI 的优势与局限

**AI 的优势：**

- **数学推导能力强：**能快速推导韦达定理、面积公式、离心率范围等
- **代码模式化：**每种题型的代码结构相似，AI 擅长模式化生成

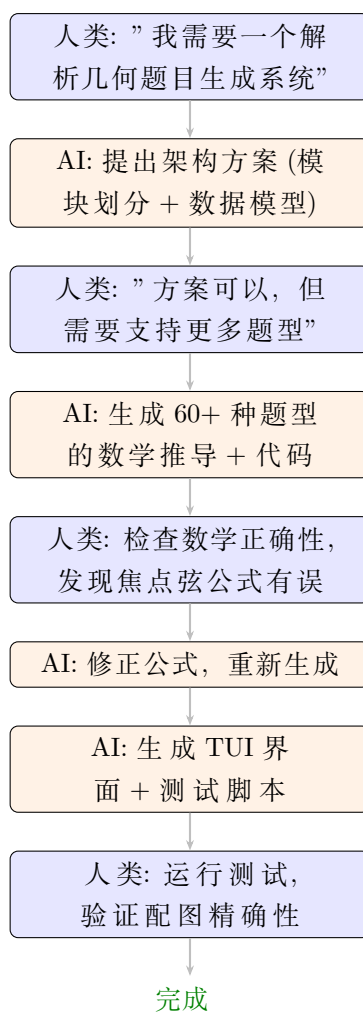


图 2: AI 辅助开发工作流

- **LaTeX 模板生成**: 自动生成格式严谨的 LaTeX 题干和解答
- **测试覆盖全面**: 能快速生成覆盖所有题型的测试脚本

**AI 的局限:**

- **数学正确性需验证**: 部分复杂推导 (如阿基米德三角形面积公式) 可能有错误
- **边界条件处理**: 需要人类检查判别式为负时的异常处理
- **设计审美**: UI 设计需要人类把关最终效果
- **领域知识**: 对于高考/竞赛的具体出题规范, 需要人类指导

## 8.5 智能体应用场景

**已实现:**

- **TUI 解析智能体**—用户输入中文描述, 系统自动匹配知识点和题型

**规划中:**



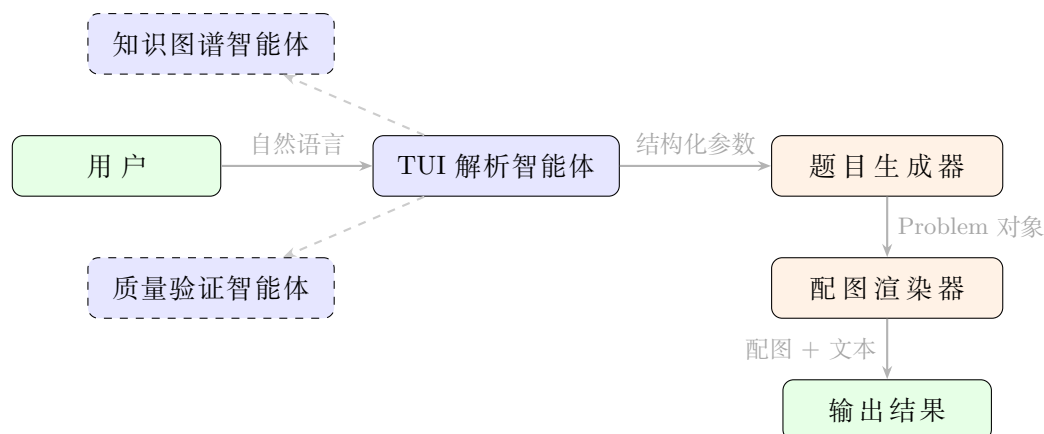


图 3: 智能体应用场景架构图

- **质量验证智能体**—自动检查判别式、参数合理性、LaTeX 语法
- **知识图谱智能体**—构建圆锥曲线知识点图谱，推荐相关题型
- **对话式出题智能体**—接入 LLM API，支持多轮对话式出题

## 9 测试与验证

### 9.1 全题型测试

`test_all_types.py` 对所有 60 种题型进行自动化测试：

1. 题目生成—不抛异常
2. 题干验证—非空、长度 > 20 字符
3. 解答验证—非空、长度 > 20 字符
4. 答案验证—非空
5. LaTeX 渲染—输出非空
6. 配图渲染—文件存在、大小 > 1KB
7. 关键点验证—至少 1 个关键点

### 9.2 测试结果

总计：60 种题型

通过：60

失败：0

通过率：100.0%

### 9.3 输出文件

output/Question\_YYYYMMDD\_HHMMSS/

|              |                          |
|--------------|--------------------------|
| diagram.png  | # 配图 (42-98 KB, 150 DPI) |
| problem.tex  | # LaTeX 题干               |
| solution.tex | # LaTeX 解答               |
| problem.txt  | # Unicode 纯文本            |